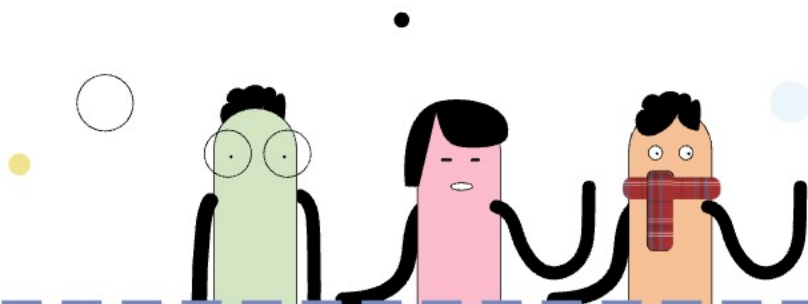




GENAI PRODUCTION

Indepth

Temperature

Python

```
temperature=0
```

means:

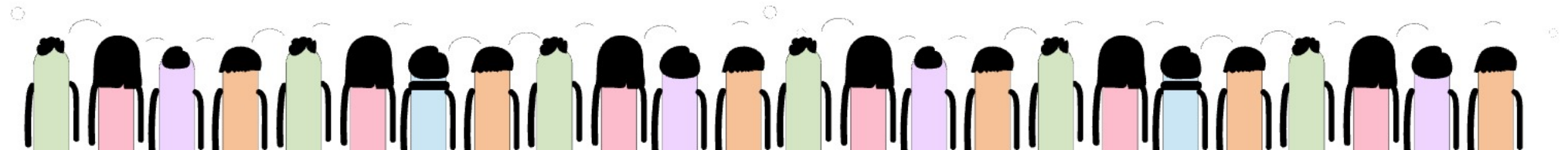
Give stable deterministic answers

SIMPLE FLOW

```
graph TD
    A[Low Temperature] --> B[Stable + Predictable]
    C[High Temperature] --> D[Creative + Random]
```

SIMPLE IDEA

Temperature	Behavior
0	predictable
0.2	focused
0.5	balanced
1.0	creative
2.0	highly random



Maxtokens



`max_tokens` controls:

Maximum size of AI response

REAL INDUSTRY USAGE

Industry	Why <code>max_tokens</code> important
Customer Support	short responses
CloudOps	concise remediation
Banking	controlled explanations
Cybersecurity	fast incident analysis

SIMPLE EXAMPLE

`max_tokens = 20`

Output:

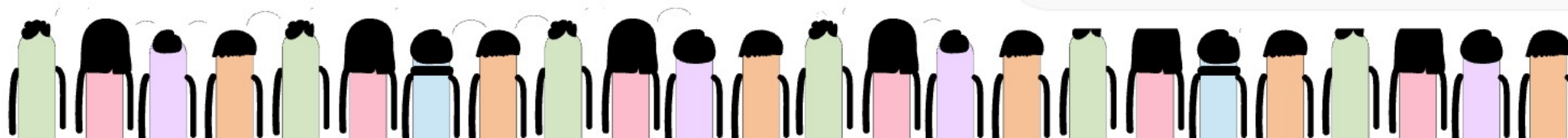
API server overloaded due to CPU spike.

Very short.

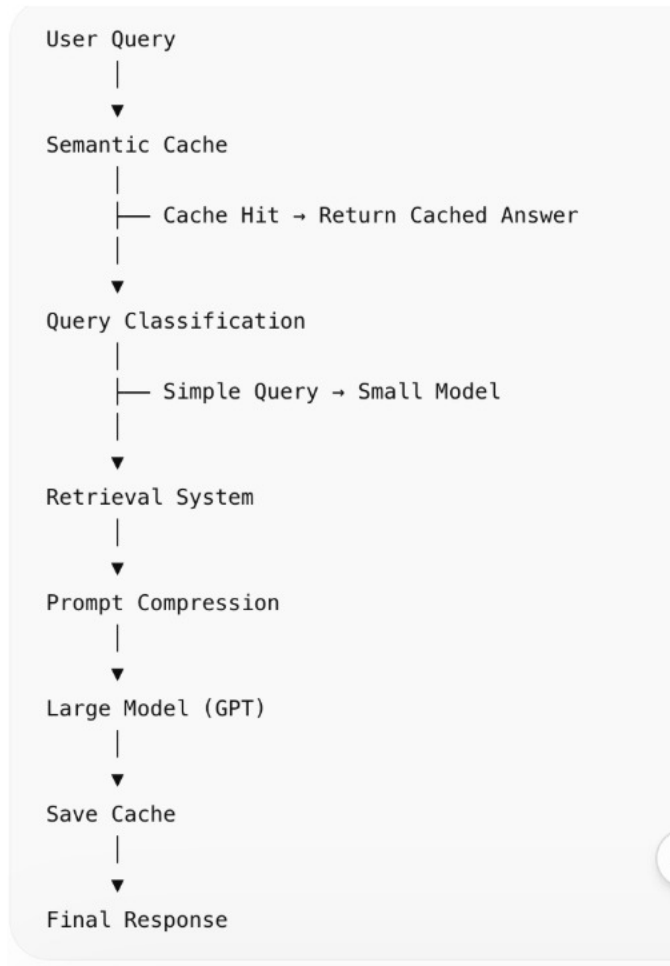
`max_tokens = 500`

Output:

Long detailed infrastructure explanation...

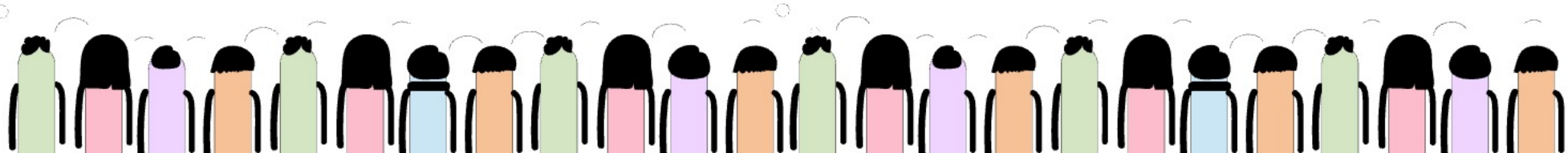


Cost Saving



SIMPLE IDEA

Score	Meaning
1.0	very similar
0.8	related
0.2	weak relation
0	unrelated





Cosine



Cosine Similarity — Simple Definition

Cosine similarity measures:

How similar two vectors are

using:

Angle between vectors

NOT actual distance.

WHY IMPORTANT?

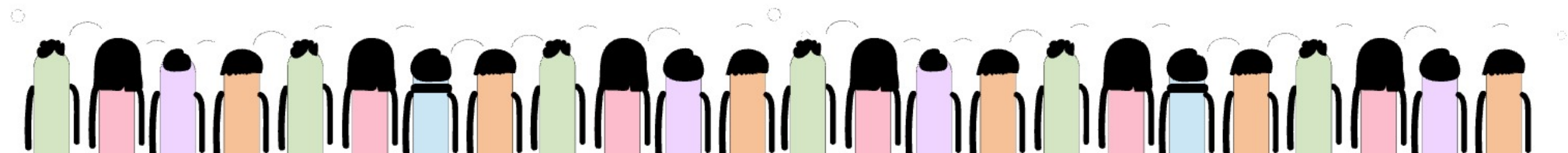
In AI systems:

Text → converted into vectors

Then we need to know:

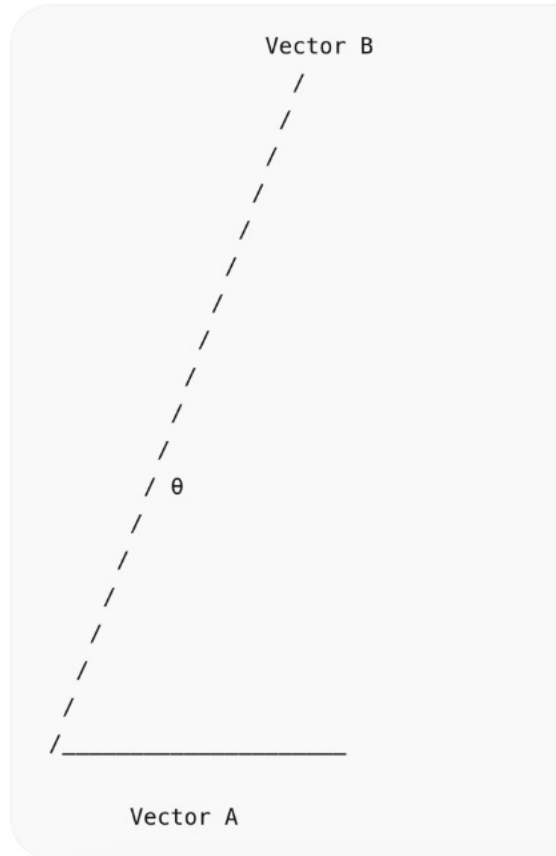
Which vectors have similar meaning?

Cosine similarity solves this.



Cosine

SIMPLE DIAGRAM

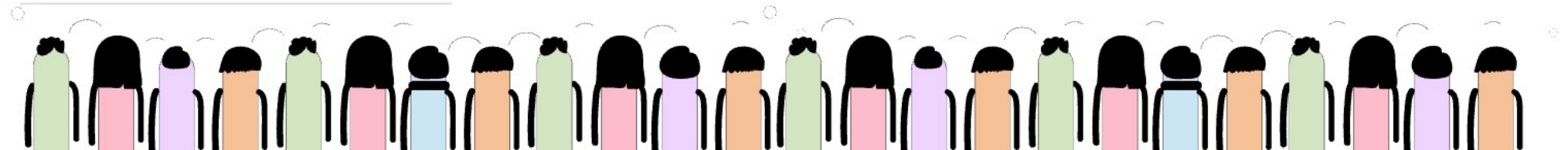


WHAT IS θ (THETA)?

Angle between vectors.

IMPORTANT LOGIC

Angle	Similarity
0°	very similar
90°	unrelated
180°	opposite





Cosine



COSINE SIMILARITY SCORES

Score	Meaning
1.0	identical meaning
0.8	highly related
0.5	somewhat related
0	unrelated
-1	opposite meaning

Query

"reduce API latency"

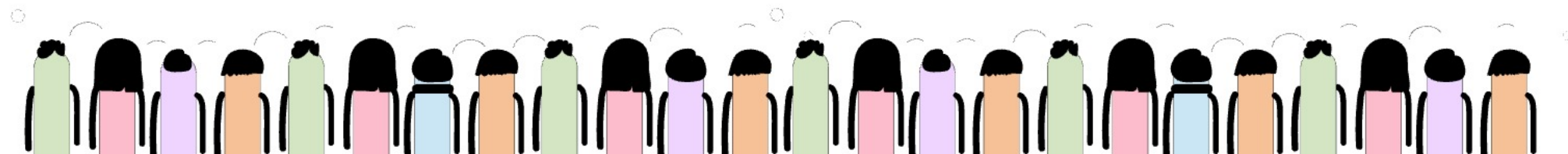
Document 1

"improve API performance"

Cosine similarity:

0.91

Very similar meaning.



Cosine

Query

"reduce API latency"

Document 1

"improve API performance"

Cosine similarity:

0.91

Very similar meaning.

Document 2

"Kafka consumer lag"

Cosine similarity:

0.22

Weak relationship.

1 RAG SYSTEMS

User Query



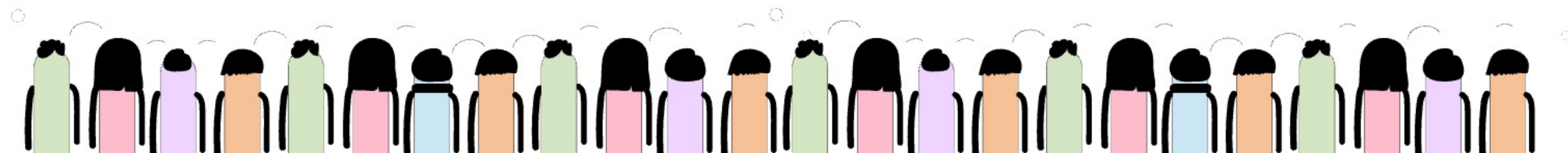
Embedding Vector



Cosine Similarity



Most Relevant Documents





ANN Vector Index



STEP 1 — RAW DOCUMENTS

Example:

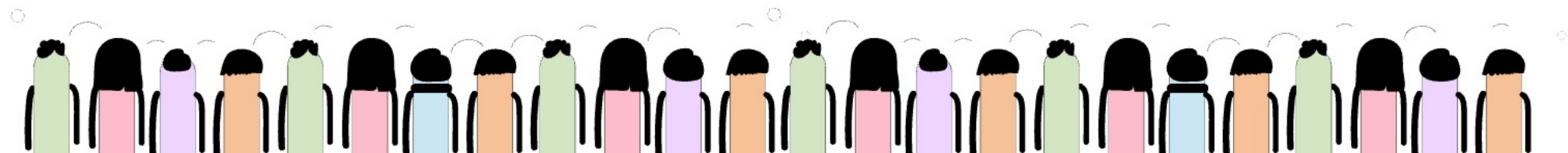
```
"Kubernetes scaling"  
"API latency"  
"Kafka streaming"
```

STEP 2 — EMBEDDING MODEL

Converts text into vectors.

Example:

```
</> Python  
  
[  
  [0.11, 0.91, ...],  
  [0.55, 0.33, ...],  
  [0.88, 0.12, ...]  
]
```





ANN Vector Index



STEP 3 — ANN INDEX CREATION

ANN algorithm organizes vectors into:

- ✓ clusters
- ✓ graphs
- ✓ trees
- ✓ partitions

depending on algorithm.

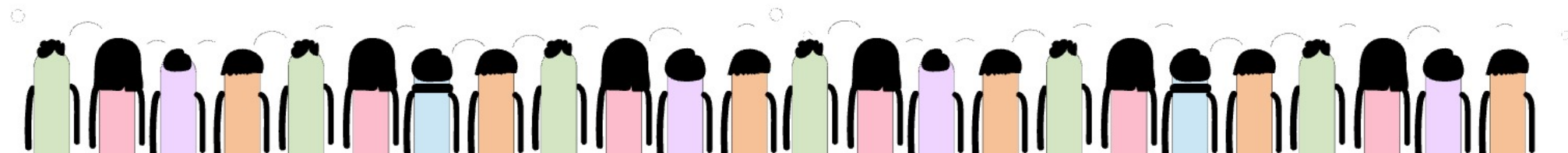
SIMPLE IDEA

Instead of:

searching ALL vectors

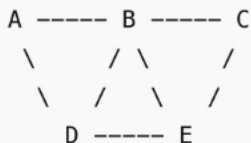
search only:

likely nearby vectors



ANN Vector Index

INTERNAL GRAPH STRUCTURE



Each node:

- vector
- connected to similar vectors

WHY THIS POWERFUL?

During search:

Jump through nearby nodes

instead of scanning all vectors.

SEARCH FLOW

Query Vector



Enter Graph

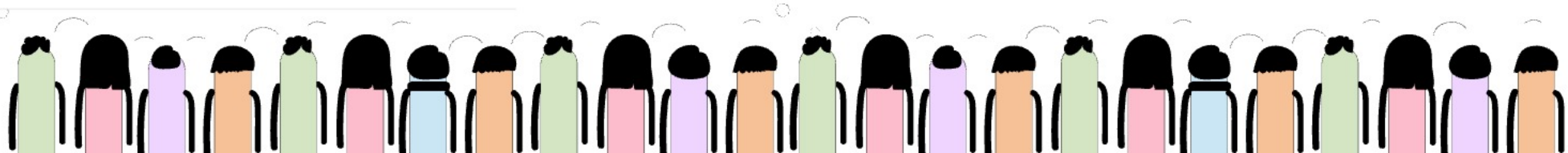


Traverse Nearby Nodes



Find Closest Vectors

VERY fast.



ANN Vector Index



SIMPLE INDUSTRY VERSION OF YOUR CODE

Instead of:

```
<> Python  
  
cosine_similarity(  
    query_embedding,  
    cache_embeddings  
)
```

industry uses:

```
<> Python  
  
vector_db.search(  
    query_embedding,  
    top_k=5  
)
```

because DB already:

- indexed
- optimized
- distributed



WITHOUT ANN

Query



Compare against:

1

2

3

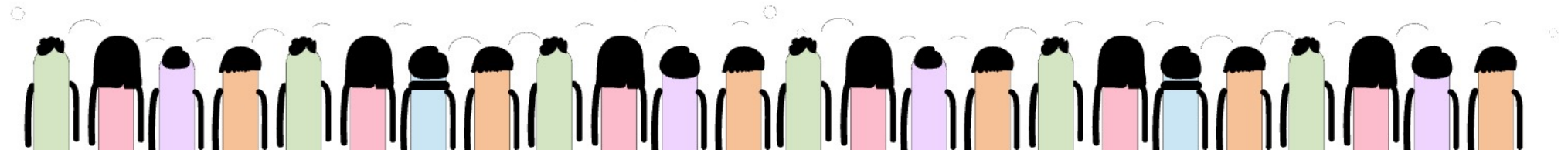
4

5

...

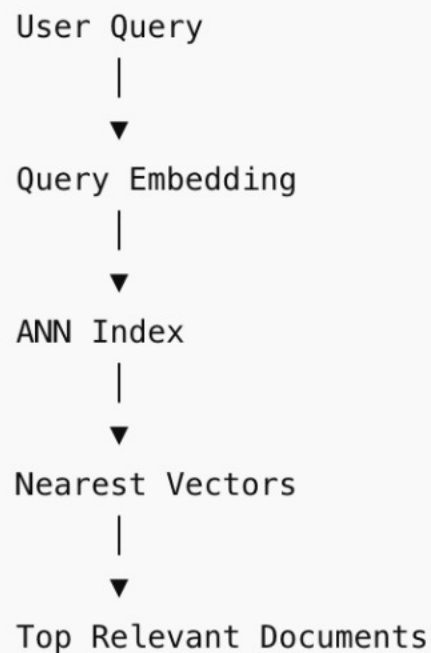
10,000,000 vectors

Very expensive.



ANN Vector Index

REAL ENTERPRISE FLOW



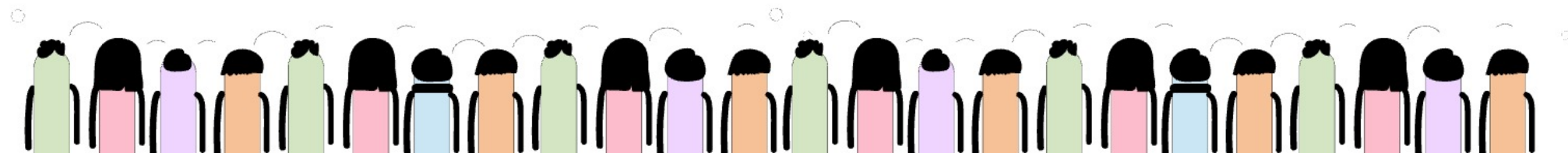
ANN Index Creation — Internal Explanation

ANN Index Creation means:

Building optimized search structure
for vectors

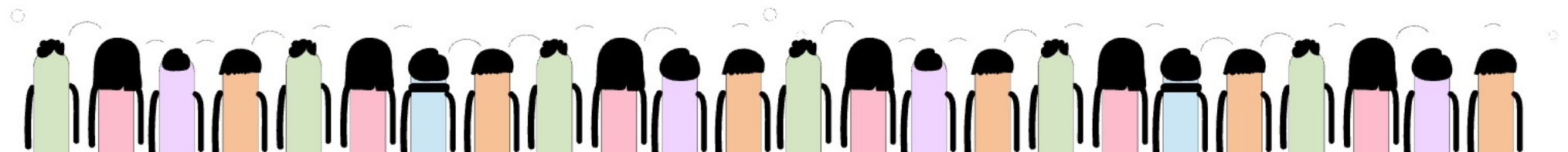
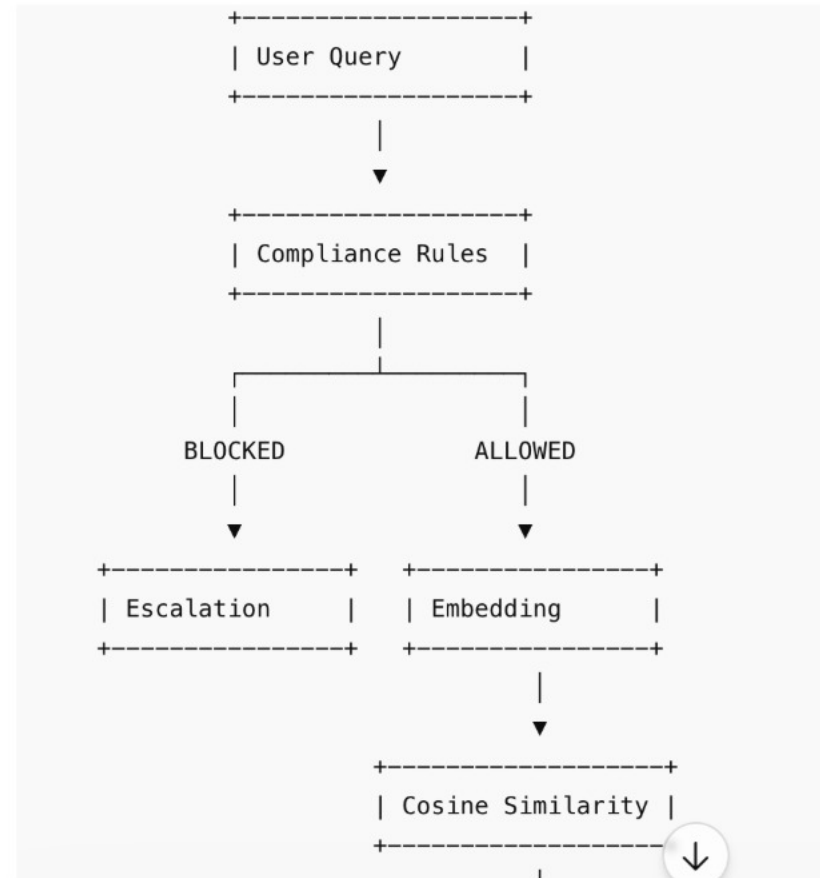
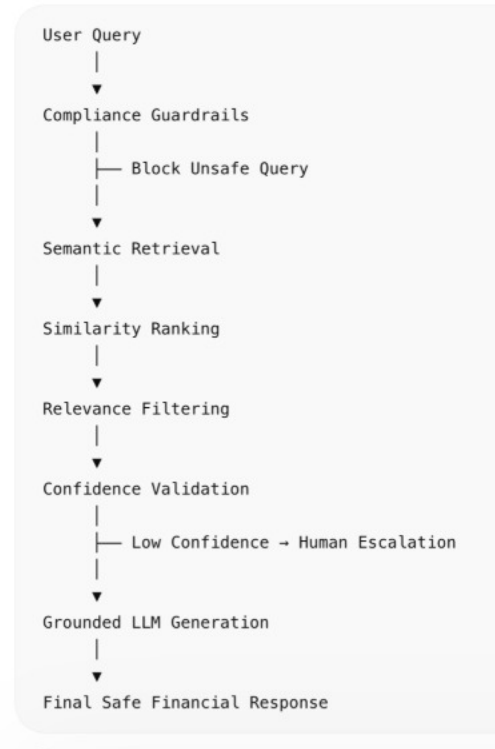
so retrieval becomes:

VERY fast

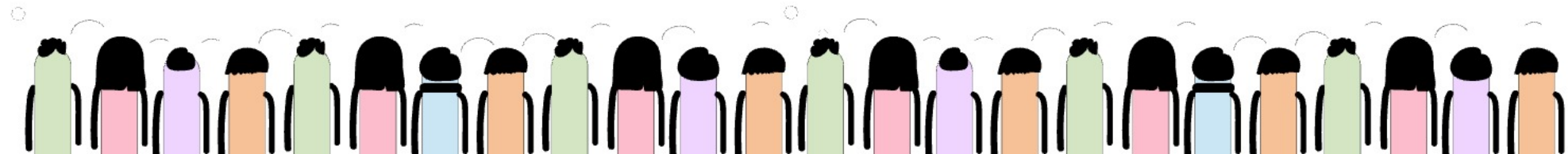
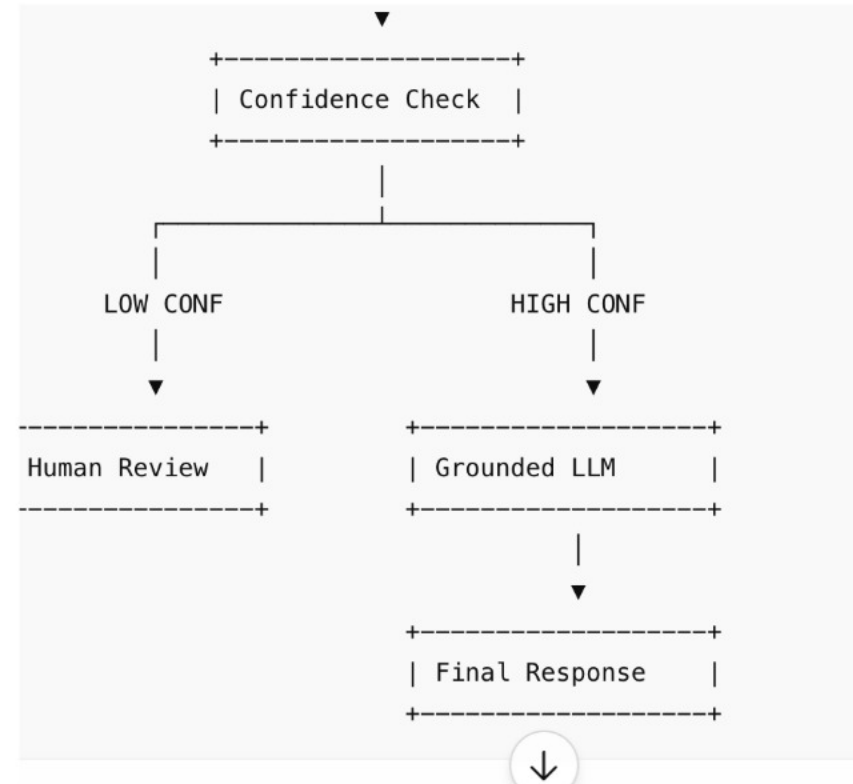
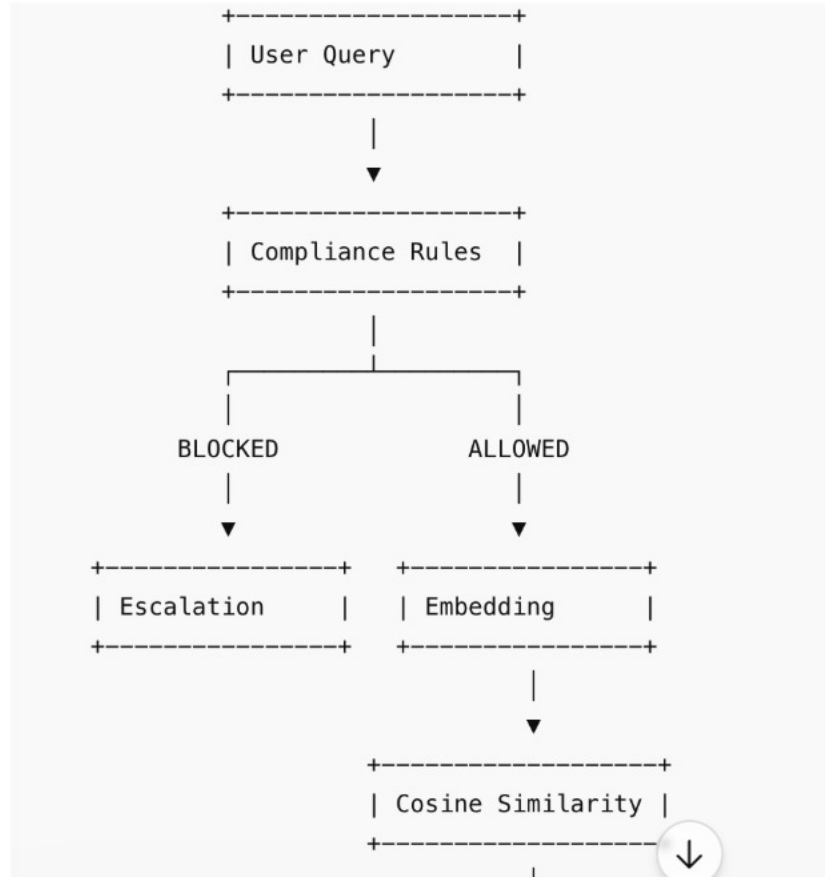


Financial Compliance AI Pipeline

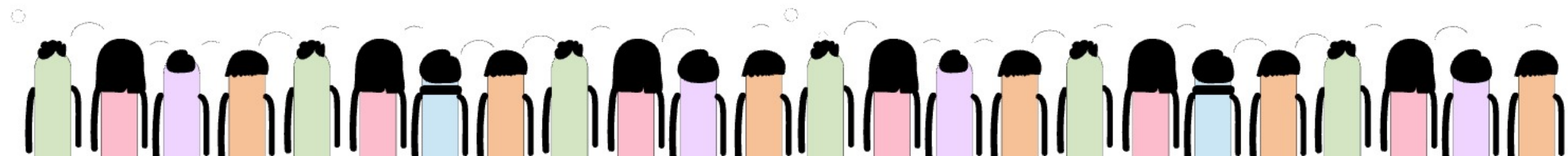
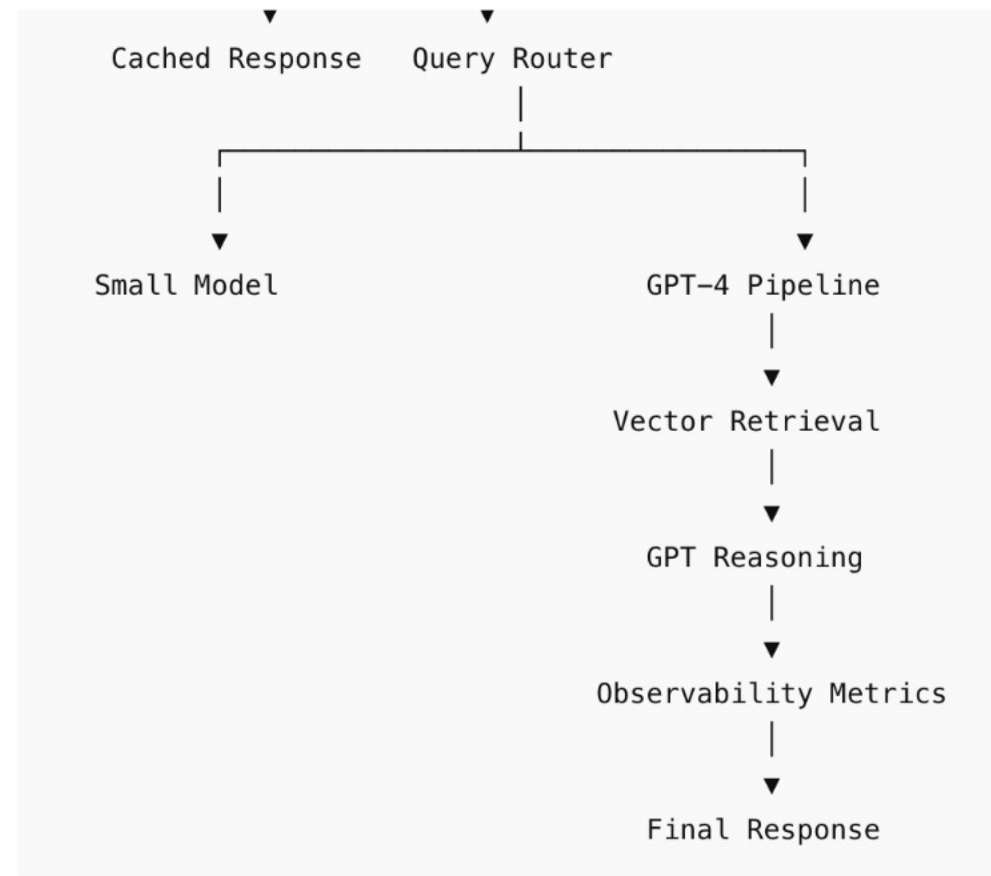
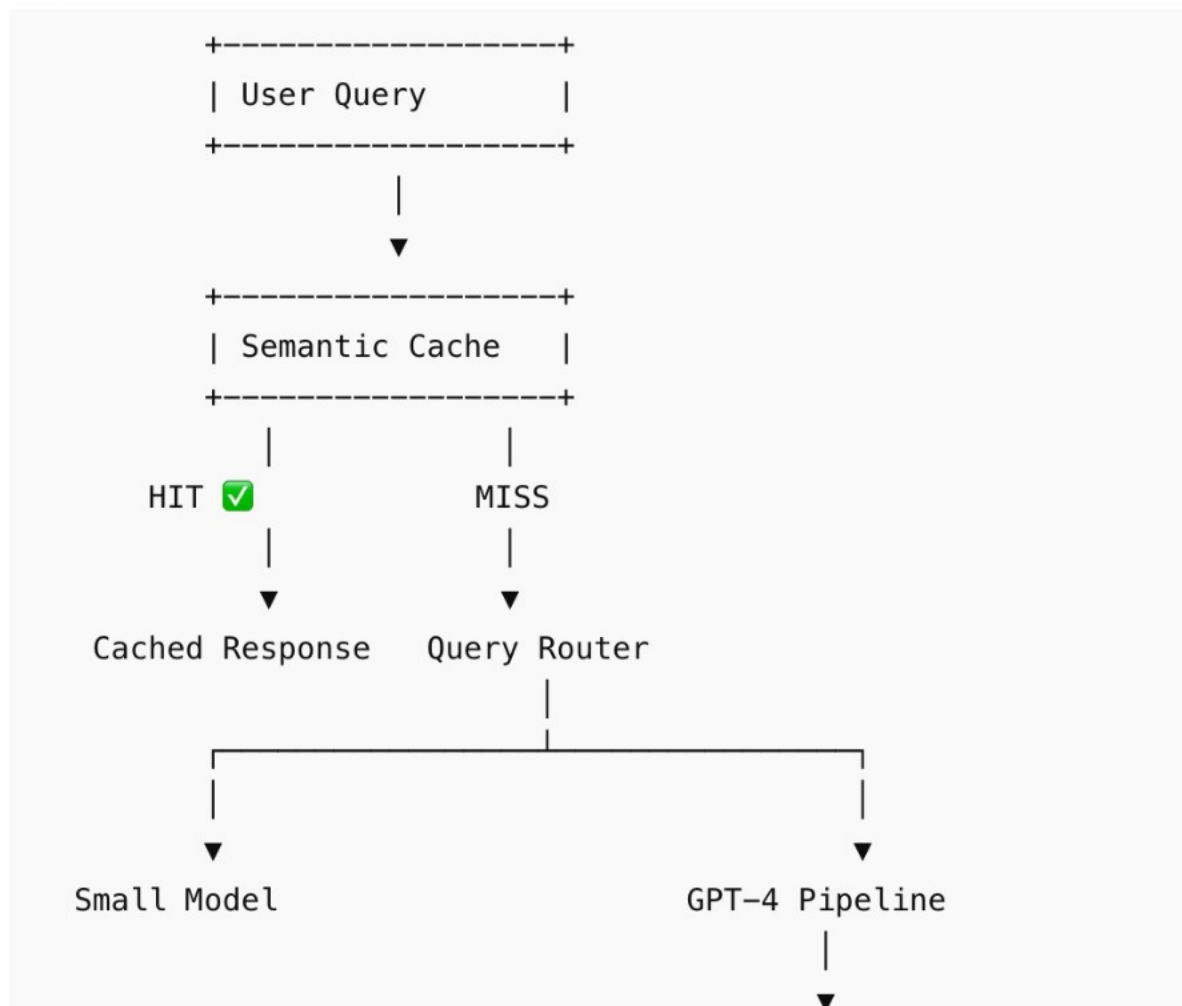
COMPLETE SYSTEM FLOW



Financial Compliance AI Pipeline



Observably



AI Efficiency

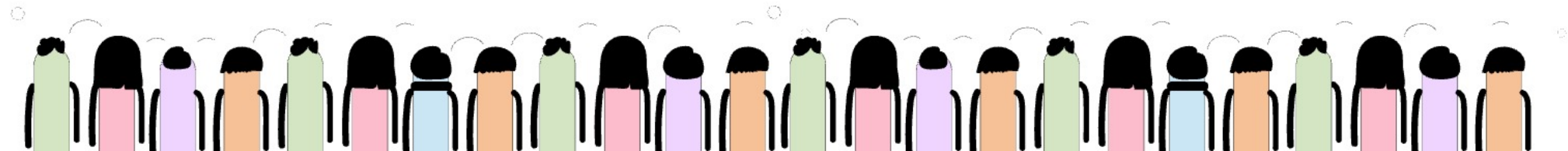
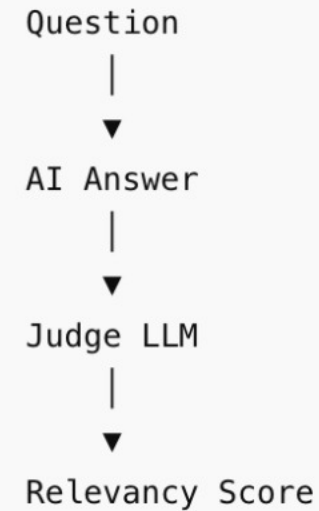
1 `AnswerRelevancyMetric()`

SIMPLE PURPOSE

Checks:

Did AI answer the actual question?

INTERNAL FLOW





AI Efficiency



INTERNALLY WHAT HAPPENS?

Framework builds evaluation prompt like:

QUESTION:

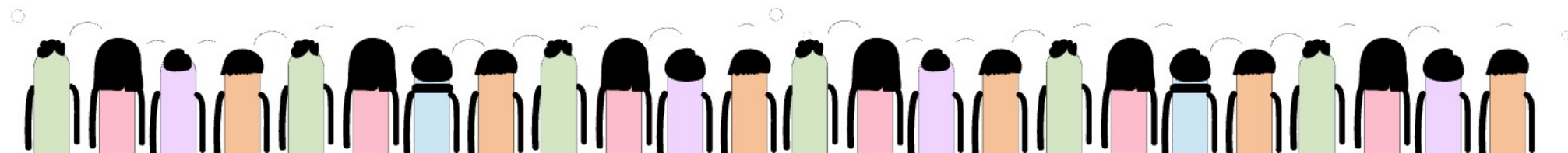
Why Kubernetes API slow?

ANSWER:

Kubernetes uses containers.

Evaluate:

Does answer directly address question?



AI Efficiency

2 HallucinationMetric()

SIMPLE PURPOSE

Checks:

Did AI invent facts
not present in context?

Source Context



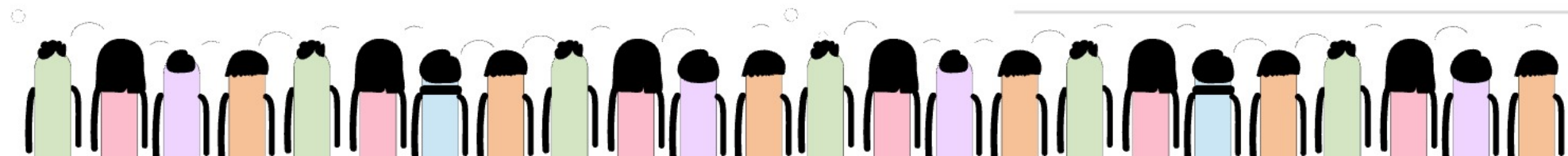
AI Answer



Judge LLM



Hallucination Score



AI Efficiency

INTERNALLY WHAT HAPPENS?

Framework creates evaluation prompt:

CONTEXT:

API outage caused by CPU overload.

ANSWER:

Fiber cable failure caused outage.

Does answer contain information
unsupported by context?

RELEVANCY

Question



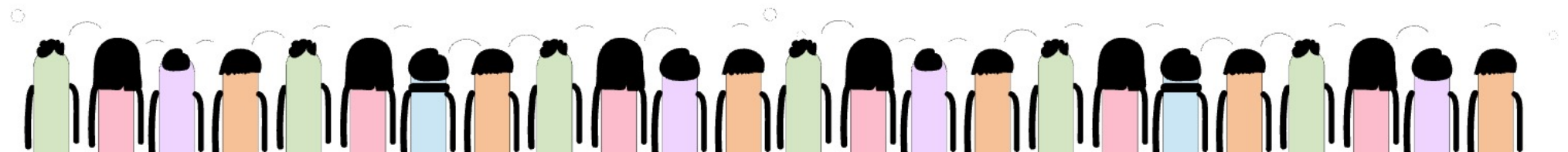
AI Answer



Semantic Alignment Check

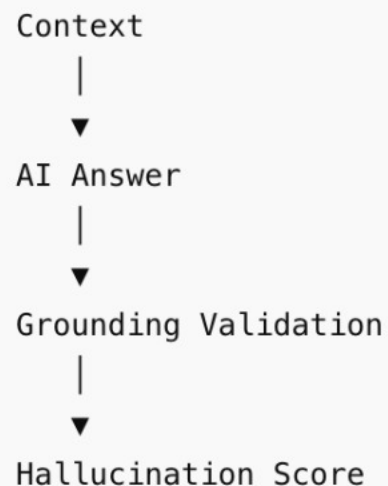


Relevancy Score



AI Efficiency

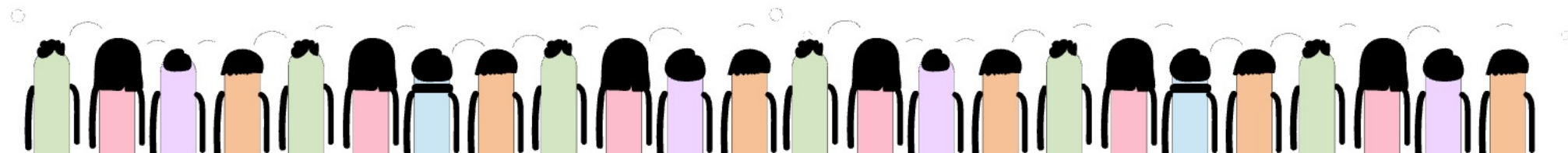
HALLUCINATION



FINAL SIMPLE UNDERSTANDING

`LLMTestCase(...)` packages:

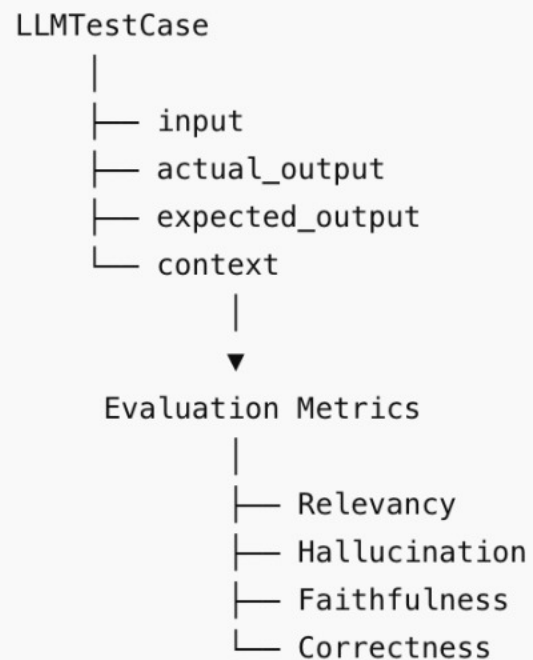
Question
+
AI answer
+
Expected answer
+
Grounding context



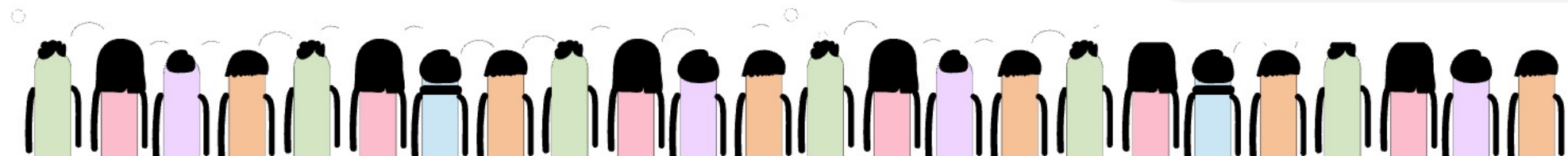
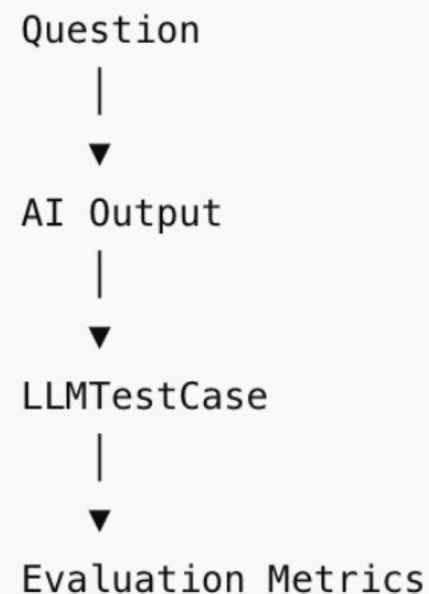
AI Efficiency



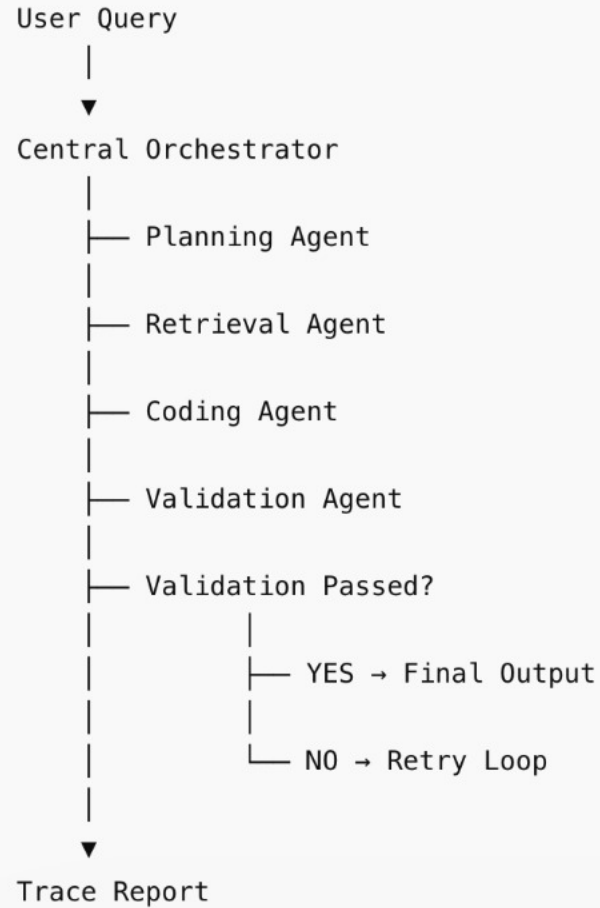
COMPLETE EVALUATION FLOW



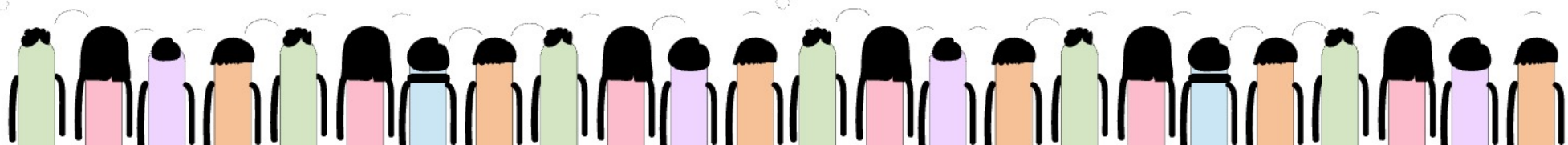
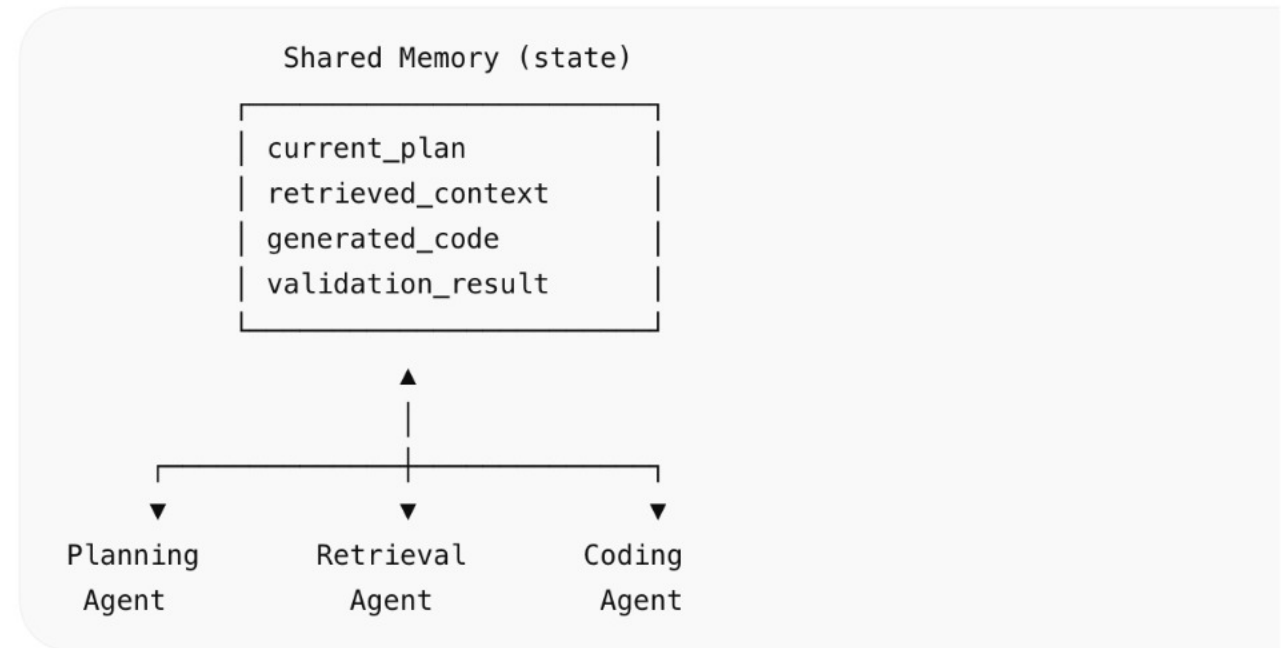
SIMPLE FLOW



Multi Agent



COMPLETE COMMUNICATION ARCHITECTURE





Multi Agent



INTERNAL MEMORY STRUCTURE

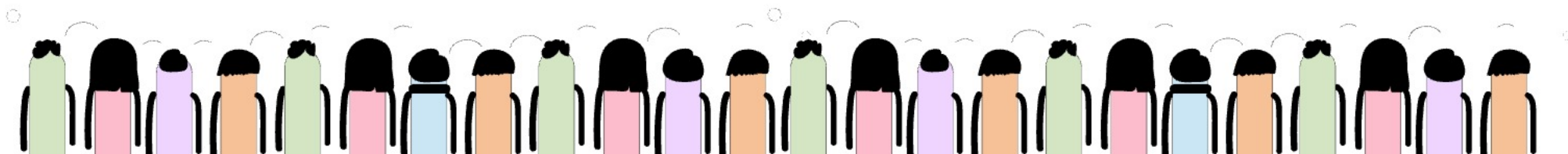
Initially:

</> Python

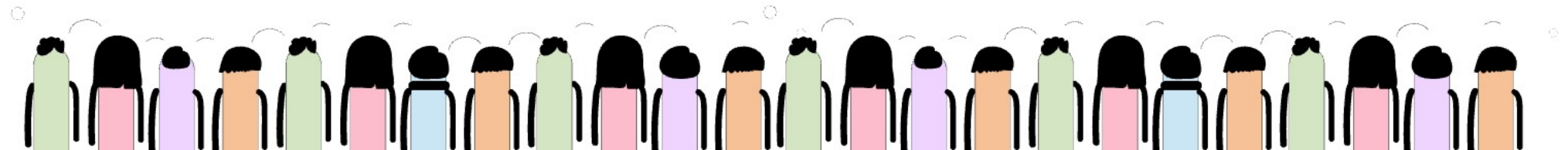
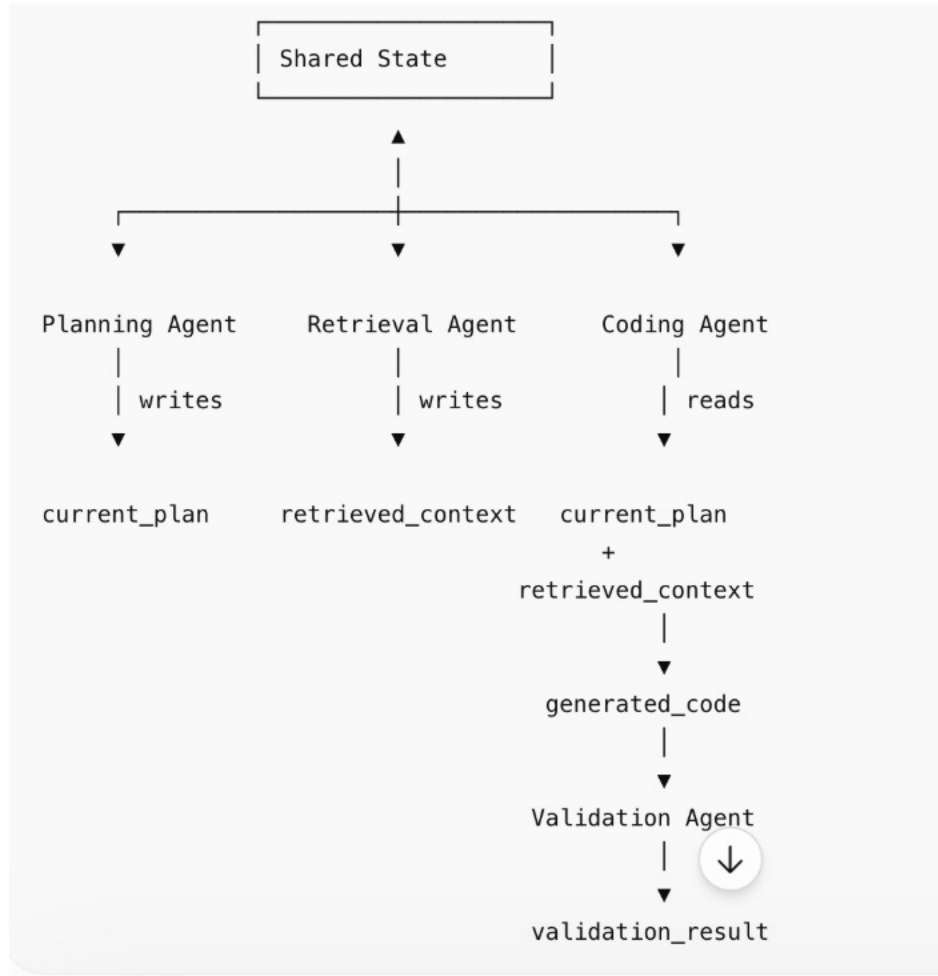
```
state = {  
  
    user_query:  
        "Build scalable API latency module",  
  
    current_plan:  
        "",  
  
    retrieved_context:  
        "",  
  
    generated_code:  
        ""  
}
```

SAME PATTERN USED EVERYWHERE

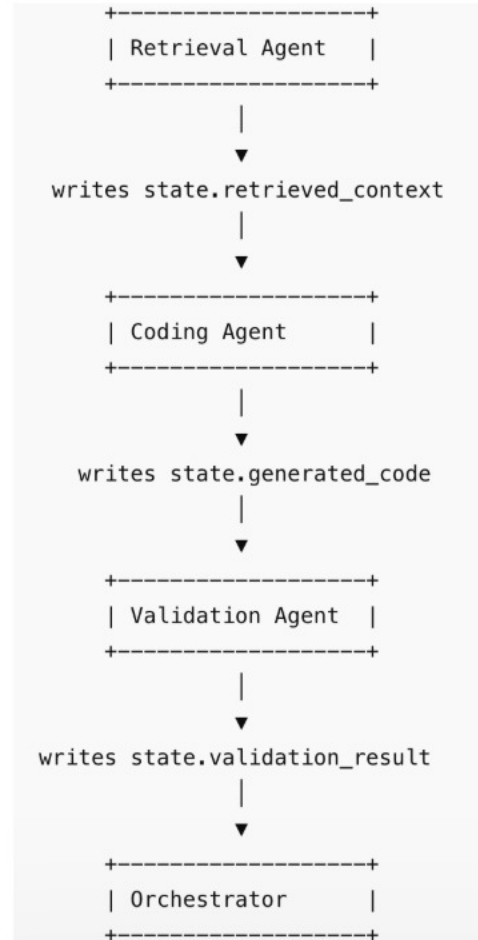
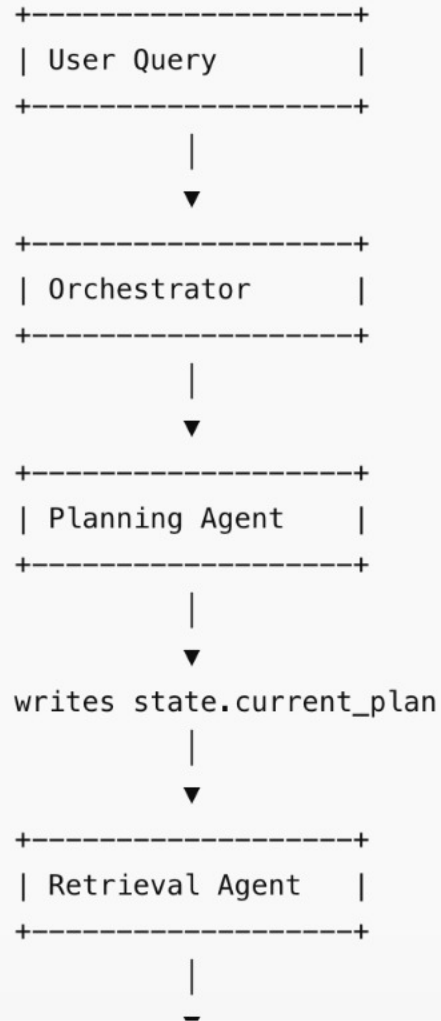
Field	Written By	Read By
current_plan	planning_agent	coding_agent
retrieved_context	retrieval_agent	coding_agent
generated_code	coding_agent	validation_agent
validation_result	validation_agent	orchestrator



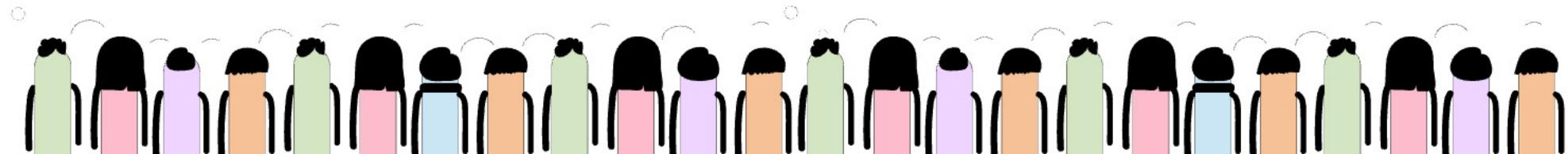
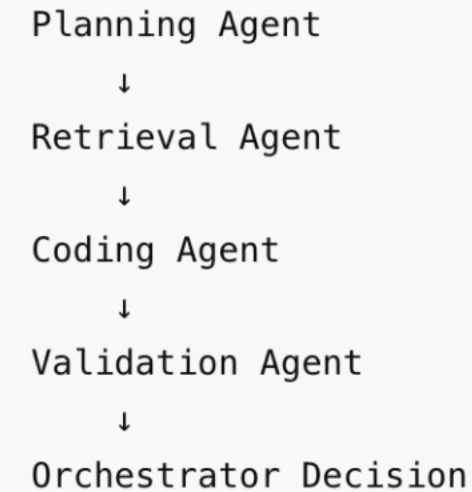
Multi Agent



Multi Agent



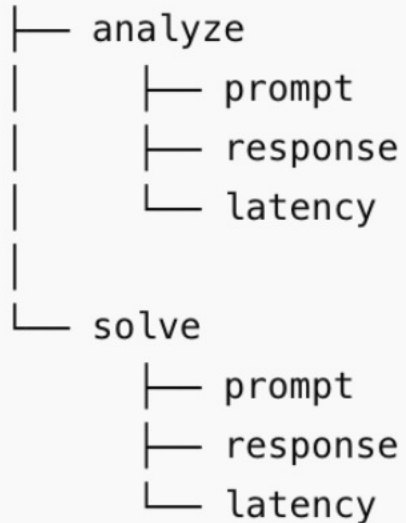
Agents execute in this exact sequence:



LangFuse

WHAT YOU WILL SEE

Trace ID



REAL ENTERPRISE FLOW

LangGraph Workflow

LLM Calls

Langfuse Tracing

Dashboard Observability

